



PREMIER UNIVERSITY

Department of Computer Science & Engineering

Lab Report

Report No. : 10

Course Code : CSE 454

Course Title : Compiler Construction Laboratory

Report Title : Write a program to construct a predictive parsing table for a context-free grammar..

Date of Submission : 10/21/2025

Submitted To:

Ms. Tanni Dhoom,
Assistant Professor,
Department of Computer Science and Engineering.

Submitted By:

Name	: MD Habibul Basher Saikat
ID	: 2104010202254
Program	: B.Sc. in CSE
Batch	: 40 th
Section	: C

<u>Remarks:</u>

Experiment No: 10

Experiment Name: Write a program to construct a predictive parsing table for a context-free grammar.

Objectives:

- Learn what a predictive (LL(1)) parsing table is and why it is useful.
- Build the LL(1) parsing table from FIRST and FOLLOW sets.
- Detect table conflicts that show the grammar is not LL(1).
- Prepare the grammar (remove left recursion, left factor) so table construction works well.

Algorithm:

Prerequisites:

1. Remove left recursion from the grammar. Left recursion prevents simple top-down parsing.
2. Apply left factoring where needed. Common prefixes should be factored out.
3. Compute FIRST sets for all symbols. FIRST is needed to decide table entries.
4. Compute FOLLOW sets for all non-terminals. FOLLOW is used when productions can produce ϵ .
5. Tokenize productions into lists of symbols (not raw strings) so multi-character non-terminals are safe.

Step-by-step algorithm to build the predictive parsing table:

1. **Input:**
 - Tokenized grammar G with non-terminals N and terminals T.
 - FIRST sets and FOLLOW sets already computed.
 - Use ϵ for empty string and \$ as end-marker.
2. **Initialize table:**
 - Create an empty table M indexed by (A, a) for each $A \in N$ and each terminal $a \in T \cup \{ \$ \}$.
 - Set all entries to error (or blank).
3. **For every production $A \rightarrow \alpha$ in the grammar do:**
 - a. Compute FIRST(α) (the FIRST set of the sequence α):
 - Add FIRST(X_1) minus ϵ . If X_1 can produce ϵ , include FIRST(X_2), etc. If all symbols of α can produce ϵ , then $\epsilon \in \text{FIRST}(\alpha)$.
 - b. For each terminal a in FIRST(α) except ϵ :
 - If M[A, a] is empty, set M[A, a] = α (store the production).
 - If M[A, a] is already set with a different production, record a **conflict** (grammar is not LL(1)).
 - c. If $\epsilon \in \text{FIRST}(\alpha)$ then for each terminal b in FOLLOW(A):
 - If M[A, b] is empty, set M[A, b] = α (the ϵ -production).
 - If M[A, b] already has a different production, record a **conflict**.

4. **Fill sync / error entries (optional):**

- For error recovery, some implementations put a special sync entry for (A, b) when $b \in \text{FOLLOW}(A)$ and the entry is blank. This helps in simple panic-mode recovery.

5. **Finish:**

- After all productions, the table M is ready.
- If any conflicts were recorded, the grammar is **not LL(1)** and needs change (left factoring, removing ambiguity, or refactoring).
- Otherwise, M can be used by a top-down predictive parser.

6. **Using the table in parsing (brief):**

- Start with stack [S, \$] where S is start symbol and input ends with \$.
- Read top of stack and current input symbol a. If top is terminal and equals a, pop and advance. If top is non-terminal A, consult M[A, a]. If it gives a production α , pop A and push α (rightmost symbol on top). If M[A, a] is error, report parse error or try recovery.
- Repeat until stack is empty and input consumed.

Code Snippet:



```
ccl_8_2254_main.py x Test_case.txt
Compiler_construction > ccl_8_2254_main.py > detect_left_recursion
1  from collections import defaultdict
2
3  # ----- Left recursion functions -----
4  def detect_left_recursion(rules_lines):
5      print("\n--- Left Recursion Detection Result ---")
6      left_recursive_count = 0
7      parsed_rules = []
8
9  > for rule in rules_lines: ...
27
28      print(f"\nTotal rules with left recursion: {left_recursive_count}")
29      return parsed_rules, left_recursive_count
30
31
32  def remove_left_recursion(parsed_rules):
33      """
34      Removes left recursion from grammar using standard algorithm
35      A → Aα | β becomes A → βA' and A' → αA' | ε
36      """
37      print("\n--- Left Recursion Removal Result ---")
38      grammar = {}
39      new_rules = []
40
41      # First pass: identify recursive rules
42      has_recursion = False
43  > for nt, prods in parsed_rules: ...
88
89      if not has_recursion:
90          print("\nNo left recursion found; grammar unchanged.")
91          return parsed_rules
92
93      print("\n--- Grammar after left recursion removal ---")
94  > for nt, prods in new_rules: ...
96
97      return new_rules
--
```

Figure 1: Left Recursion Code Snippet

```

100 # ----- Left factoring functions -----
101 EPSILON = 'ε'
102
103 > class _TrieNode: ...
107
108 > def _choose_char_mode(productions): ...
113
114 > def _tokenize_char_mode_with_nts(prod, nonterminals): ...
137
138 > def _tokenize(prod, char_mode, nonterminals=None): ...
149
150 > def _build_trie_with_mode(productions, char_mode, nonterminals=None): ...
162
163 > def _collect_maximal_prefixes(root): ...
176
177 def detection_left_factoring(parsed_rules, verbose=True):
178     factoring_map = {}
179     total_groups = 0
180
181     for nt, prods in parsed_rules:
182         if verbose:
183             print(f"\nTotal left factoring groups in grammar: {total_groups}")
184         return factoring_map, total_groups
185
186 > def _unique_new_nt(base, existing_set): ...
217
218 def removal_left_factoring(parsed_rules):
219     print("\n--- Left Factoring Removal Result ---")
220     grammar = {nt: list(prods) for nt, prods in parsed_rules}
221     existing_nts = set(grammar.keys())
222
223     iteration = 0
224     while True:
225
226         print("\n--- Grammar after left factoring removal ---")
227         final_rules = []
228         for nt, prods in grammar.items():
229             print(f"{nt} → {' | '.join(prods)}")
230             final_rules.append((nt, prods))
231         return final_rules
232
233 ---

```

Figure 2: Left Factoring Code Snippet

```

295 # ----- FIRST sets -----
296 def build_tokenized_grammar(parsed_rules):
297     nonterminals = [nt for nt, _ in parsed_rules]
298     char_mode_map = {}
299     for nt, prods in parsed_rules:
300         char_mode_map[nt] = _choose_char_mode(prods)
301
302     grammar_tokens = {}
303     for nt, prods in parsed_rules:
304         cm = char_mode_map.get(nt, True)
305         tokenized_prods = []
306         current_nts = set([x for x, _ in parsed_rules])
307         for p in prods:
308             if cm:
309                 tokens = _tokenize(p, True, current_nts)
310             else:
311                 tokens = _tokenize(p, False, None)
312             tokenized_prods.append(tokens)
313         grammar_tokens[nt] = tokenized_prods
314
315     terminals = set()
316     for nt, prods in grammar_tokens.items():
317         for prod in prods:
318             for tok in prod:
319                 if tok == EPSILON:
320                     continue
321                 if tok not in grammar_tokens.keys():
322                     terminals.add(tok)
323     return grammar_tokens, char_mode_map, set(grammar_tokens.keys()), terminals
324
325 ---

```

```

325 def compute_first_sets(grammar_tokens):
326     FIRST = defaultdict(set)
327
328     nonterminals = set(grammar_tokens.keys())
329     symbols = set()
330     for nt, prods in grammar_tokens.items():
331         for prod in prods:
332             for tok in prod:
333                 symbols.add(tok)
334
335     terminals = set([s for s in symbols if s not in nonterminals and s != EPSILON])
336     for t in terminals:
337         FIRST[t].add(t)
338
339     for nt in nonterminals:
340         FIRST[nt] = set()
341
342     changed = True
343     while changed: ...
344     return FIRST
345
346
347 def pretty_print_first_sets(FIRST):
348     print("\n--- FIRST sets ---")
349     for sym in sorted(FIRST.keys()):
350         if sym.isprintable():
351             items = ', '.join(sorted(FIRST[sym]))
352             print(f"FIRST({sym}) = {{ {items} }}")
353
354

```

Figure 3: First Set Code Snippet

[illegible]

```

420 def pretty_print_follow_sets(FOLLOW):
421     print("\n--- FOLLOW sets ---")
422     for sym in sorted(FOLLOW.keys()):
423         if sym.isprintable():
424             items = ', '.join(sorted(FOLLOW[sym]))
425             print(f"FOLLOW({sym}) = {{ {items} }}")
426

```

Figure 4: Follow Set Code Snippet

```

428 # ----- LL(1) Parsing Table -----
429 def construct_parsing_table(grammar_tokens, FIRST, FOLLOW):
430     table = {}
431     conflicts = []
432     nonterminals = list(grammar_tokens.keys())
433     terminals = set()
434     for nt, prods in grammar_tokens.items():
435         for prod in prods:
436             for tok in prod:
437                 if tok != EPSILON and tok not in nonterminals:
438                     terminals.add(tok)
439     terminals_list = sorted(terminals)
440     all_terminals = terminals_list + ['$']
441
442     for A in nonterminals:
443         for prod in grammar_tokens[A]:
444             first_seq = first_of_sequence(prod, FIRST)
445             for a in (first_seq - {EPSILON}):
446                 key = (A, a)
447                 if key in table and table[key] != prod:
448                     conflicts.append((A, a, table[key], prod))
449                 else:
450                     table[key] = prod
451             if EPSILON in first_seq:
452                 for b in FOLLOW[A]:
453                     key = (A, b)
454                     if key in table and table[key] != prod:
455                         conflicts.append((A, b, table[key], prod))
456                     else:
457                         table[key] = prod
458     is_ll1 = len(conflicts) == 0
459     return table, is_ll1, conflicts, sorted(all_terminals)
460
461 def pretty_print_parsing_table(table, nonterminals, terminals, grammar_tokens):
462     print("\n--- LL(1) Parsing Table ---")
463
464     # Create header row
465     header = "| Non-Terminal |"
466     for term in terminals:
467         header += f" {term:>10} |"
468     print(header)
469     print("-" * len(header))
470
471     # Fill table rows
472     for nt in nonterminals:
473         row = f"| {nt:>12} |"
474         for term in terminals:
475             key = (nt, term)
476             if key in table:
477                 prod = table[key]
478                 # Convert production (list) to string
479                 if isinstance(prod, list):
480                     prod_str = ', '.join(prod)
481                 else:
482                     prod_str = str(prod)
483                 # Truncate long productions for display
484                 prod_display = prod_str if len(prod_str) <= 10 else prod_str[:7] + "..."
485                 row += f" {prod_display:>10} |"

```

```

486         else:
487             row += f" {'':>10} |"
488         print(row)
489
490     print("\n")
491
492     # Print detailed table entries (not truncated)
493     print("--- Detailed Parsing Table Entries ---")
494     for (nt, term), prod in sorted(table.items()):
495         if isinstance(prod, list):
496             prod_str = ''.join(prod)
497         else:
498             prod_str = str(prod)
499         print(f"M[{nt}, {term}] = {nt} → {prod_str}")
500
501     print("\n")
502
503 # -----
504 # MAIN PROGRAM
505 # -----
506 if __name__ == "__main__":
507     n = int(input("Enter number of rules: "))
508     rules = []
509     for _ in range(n):
510         rules.append(input().strip())
511
512     print("\n" + "=" * 70)
513     print("STEP 1: LEFT RECURSION DETECTION")
514     print("=" * 70)
515
516     # 1) Left recursion detection
517     parsed_rules, left_recursive_count = detect_left_recursion(rules)
518
519     print("\n" + "=" * 70)
520     print("STEP 2: LEFT RECURSION REMOVAL")
521     print("=" * 70)
522
523     # 2) Left recursion removal
524     rules_after_lr = remove_left_recursion(parsed_rules)
525
526     print("\n" + "=" * 70)
527     print("STEP 3: LEFT FACTORING DETECTION")
528     print("=" * 70)
529
530     # 3) Left factoring detection
531     factoring_map, total_fact_groups = detection_left_factoring(rules_after_lr, verbose=True)
532
533     print("\n" + "=" * 70)
534     print("STEP 4: LEFT FACTORING REMOVAL")
535     print("=" * 70)
536
537     # 4) Left factoring removal
538     if total_fact_groups > 0:
539         final_grammar = removal_left_factoring(rules_after_lr)
540     else:
541         print("\n--- Left Factoring Removal Result ---")
542         print("\nNo left factoring groups found; grammar unchanged after factoring pass.")
543         final_grammar = rules_after_lr
544
545     print("\n" + "=" * 70)
546     print("STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS")
547     print("=" * 70)

```

```

550     # 5) Build tokenized grammar
551     grammar_tokens, char_mode_map, nonterminals, terminals = build_tokenized_grammar(final_grammar)
552
553     # Choose start symbol
554     if final_grammar:
555         start_symbol = final_grammar[0][0]
556     else:
557         start_symbol = None
558
559     print(f"\nStart Symbol: {start_symbol}")
560     print(f"Non-terminals: {sorted(nonterminals)}")
561     print(f"Terminals: {sorted(terminals)}")
562
563     # 6) Compute FIRST sets
564     FIRST = compute_first_sets(grammar_tokens)
565     pretty_print_first_sets(FIRST)
566
567     print("\n" + "=" * 70)
568     print("STEP 6: COMPUTE FOLLOW SETS")
569     print("=" * 70)
570
571     # 7) Compute FOLLOW sets
572     FOLLOW = compute_follow_sets(grammar_tokens, FIRST, start_symbol)
573     pretty_print_follow_sets(FOLLOW)
574
575     print("\n" + "=" * 70)
576     print("STEP 7: CONSTRUCT LL(1) PARSING TABLE")
577     print("=" * 70)
578
579     # 8) Construct and print parsing table
580     table, is_ll1, conflicts, terminals_sorted = construct_parsing_table(grammar_tokens, FIRST, FOLLOW)
581     pretty_print_parsing_table(table, list(grammar_tokens.keys()), terminals_sorted, grammar_tokens)
582
583     print("=" * 70)
584     print("FINAL ANALYSIS")
585     print("=" * 70)
586     print(f"\nIs this grammar LL(1)? {is_ll1}")
587     if conflicts:
588         print(f"Number of conflicts: {len(conflicts)}")
589         print("\nConflicts found:")
590         for A, a, prod1, prod2 in conflicts:
591             prod1_str = ''.join(prod1) if isinstance(prod1, list) else str(prod1)
592             prod2_str = ''.join(prod2) if isinstance(prod2, list) else str(prod2)
593             print(f"    M[{A}, {a}]: {A} → {prod1_str} vs {A} → {prod2_str}")
594     else:
595         print("No conflicts - grammar is LL(1) compliant!")
596     print("=" * 70)

```

Output Snippet:

```

PS C:\Users\Habib\Desktop> & "C:\Program Files\Python314\python.exe" c:/L
Enter number of rules: 3
E->E+T|T
T->T*F|F
F->(E)|id

```

```

=====
STEP 1: LEFT RECURSION DETECTION
=====

```

```

--- Left Recursion Detection Result ---
E has 1 left recursive production(s):
E → E+T
T has 1 left recursive production(s):
T → T*F
F has no left recursion.

```

```

Total rules with left recursion: 2

```


=====

STEP 2: LEFT RECURSION REMOVAL

=====

--- Left Recursion Removal Result ---

Removing left recursion from E:
Left recursive productions: E+T
Non-recursive productions: T
Introduced E':
New $E \rightarrow TE'$
 $E' \rightarrow +TE', \epsilon$

Removing left recursion from T:
Left recursive productions: T*F
Non-recursive productions: F
Introduced T':
New $T \rightarrow FT'$
 $T' \rightarrow *FT', \epsilon$

--- Grammar after left recursion removal ---

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow (E) \mid id$

=====

STEP 3: LEFT FACTORING DETECTION

=====

E has no left factoring.
E' has no left factoring.
T has no left factoring.
T' has no left factoring.
F has no left factoring.

Total left factoring groups in grammar: 0

=====

STEP 4: LEFT FACTORING REMOVAL

=====

--- Left Factoring Removal Result ---

No left factoring groups found; grammar unchanged after factoring pass.

=====

STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS

=====

Start Symbol: E
Non-terminals: ['E', "E'", 'F', 'T', "T'"]
Terminals: ['(', ')', '*', '+', 'd', 'i']

--- FIRST sets ---

$\text{FIRST}(() = \{ (\}$
 $\text{FIRST}() = \{) \}$
 $\text{FIRST}(*) = \{ * \}$
 $\text{FIRST}(+) = \{ + \}$
 $\text{FIRST}(E) = \{ (, i \}$
 $\text{FIRST}(E') = \{ + , \epsilon \}$
 $\text{FIRST}(F) = \{ (, i \}$
 $\text{FIRST}(T) = \{ (, i \}$
 $\text{FIRST}(T') = \{ * , \epsilon \}$
 $\text{FIRST}(d) = \{ d \}$
 $\text{FIRST}(i) = \{ i \}$

=====

STEP 6: COMPUTE FOLLOW SETS

=====

--- FOLLOW sets ---

$\text{FOLLOW}(E) = \{ \$,) \}$
 $\text{FOLLOW}(E') = \{ \$,) \}$
 $\text{FOLLOW}(F) = \{ \$,) , * , + \}$
 $\text{FOLLOW}(T) = \{ \$,) , + \}$
 $\text{FOLLOW}(T') = \{ \$,) , + \}$

=====

STEP 7: CONSTRUCT LL(1) PARSING TABLE

=====

--- LL(1) Parsing Table ---

Non-Terminal	\$	(	)	*	+	d	i
E		TE'					TE'
E'					+TE'		
T		FT'					FT'
T'				*FT'			
F		(E)					id

--- Detailed Parsing Table Entries ---

$M[E, (] = E \rightarrow TE'$
 $M[E, i] = E \rightarrow TE'$
 $M[E', \$] = E' \rightarrow$
 $M[E',)] = E' \rightarrow$
 $M[E', +] = E' \rightarrow +TE'$
 $M[F, (] = F \rightarrow (E)$
 $M[F, i] = F \rightarrow id$
 $M[T, (] = T \rightarrow FT'$
 $M[T, i] = T \rightarrow FT'$
 $M[T', \$] = T' \rightarrow$
 $M[T',)] = T' \rightarrow$
 $M[T', *] = T' \rightarrow *FT'$
 $M[T', +] = T' \rightarrow$

=====

FINAL ANALYSIS

=====

Is this grammar LL(1)? True
 No conflicts - grammar is LL(1) compliant!

Discussion: In this experiment, I wrote a program to construct a predictive parsing table for a given context-free grammar. The goal of the table is to help the parser decide which production rule to use based on the current input symbol. To build this table correctly, the grammar first needs to be simplified by removing left recursion and applying left factoring. Then, I used the previously calculated FIRST and FOLLOW sets to fill the table entries. For each production, terminals from the FIRST set were used to place that production in the table. If any production had an ϵ (epsilon), I also used the FOLLOW set of the non-terminal to complete the table. The final table shows, for every non-terminal and terminal pair, which production should be used. During the experiment, I also learned that if two productions point to the same table cell, the grammar is not LL(1) and needs to be modified. This helped me understand how predictive parsing works and why it requires grammars without ambiguity. Overall, this experiment made the concept of LL(1) parsing and top-down parsing much clearer and practical through implementation.